

ThaiSLM — Architecture v7 (camera → landmarks → signs → Thai sentence → speech)

Scope: how v7 is built end to end, written to be reused in the project report. Changes from v6 are marked **[v7]**.
Results: `result_v7.md`. Representation space: `latent_space_v7.md`. File map: `file_structure.pdf`.

1. System at a glance



Syntax error in text mermaid version 10.9.1

The same `holistic_landmarker.task` file runs in the browser and in Python (training data and server-side uploads), so the model sees the same kind of landmarks everywhere **[v7]**.

2. Data

source	content	clips (primary)	role
TTRS dictionary	national dictionary videos	≈ 5.0k	train / val / test (signer units)
th_sl (th-sl.com)	NADT dictionary, 184 topics	≈ 7.7k	train; val and test = one unseen signer each
TSL-ONE-S	29 signers × 184 glosses (landmarks)	≈ 4.1k	train; val 3 / test 5 unseen signers
TSL51 experts + researcher	51 signs, 76 sentence templates	≈ 1.6k + 252 sentences	researcher = test signer; templates split tune / test
YouTube continuous	sentence lessons (labelled spans)	12 signers	train (transitions, sentences)
YouTube word lessons [v7]	577 harvested clips, 25 people	577	train 401 / val 90 / AGENT_test 86
data_test	5 real phone / web videos	5	never used for any decision

Totals: 19,347 primary MediaPipe clips (17.9 h), 13,573 RTMW views, 9,922 concepts, 2,074 concepts with ≥ 2 signers.

2.1 Harvest pipeline [v7] (`scripts/harvest_words.py`)

`discover` (yt-dlp search for each daily word, then crawl the channels found) → `candidates` (title parsing with pythainlp: remove lesson markers, numbers, hashtags; keep one-word titles; flag fingerspelling, questions,

phrases) → download (4 threads, data_test thumbnail check) → videos → signers (DINOv2 face embeddings; merge channels that show the same person; exclude data_test look-alikes; inherit held-out roles) → label (sign span from hand activity, consistency check against other channels, duplicate checks, roles by person). The output is items.csv, with a reason for every accepted or rejected clip.

Duplicate checks, in order: YouTube ID vs v4 → sha256 → **visual copy check** (frame thumbnails and motion energy compared at the best time offset, against same-word corpus videos and within the harvest). The pose-fingerprint check of earlier drafts could not tell a re-upload from the same person signing again, so it was replaced.

2.2 Standard Schema and handedness

133 slots (COCO-WholeBody layout), 25 fps, face points mapped to a 68-point template, short hand gaps filled. MediaPipe's 75-point output goes through schema.from_mp75. Left-dominant signers are mirrored to right-dominant (canonical_hands, decided from detected-hand activity, margin 0.62). 177 of 19,347 clips are mirrored. At inference the same rule runs per utterance, and the UI says when it mirrored.

3. Model

3.1 Sign encoder

Uni-Sign ST-GCN (body, both hands, face streams) feeding the mT5 encoder (top 8 of 12 layers trainable) and a projection head, giving a 768-d normalised embedding plus frame states H (91.9 M parameters). Initialised from Uni-Sign CSL→WLASL.

Training [v7]:

- CosFace (s 30, m 0.25) + 0.5 · SupCon.
- P 48 × K 2 batches, part of each batch made of **hard negatives**: concepts whose CosFace proxies are nearest. The neighbours are re-mined at each evaluation and written to a file that the persistent loader workers reload.
- **Cross-extractor positives**: the RTMW view of a clip counts as a positive of its MediaPipe view.
- **Webcam augmentation**: uneven 12–24 fps capture, finger jitter, short hand-detection flicker; plus speed 0.7–1.6.
- Signer retargeting p 0.3, mirror p 0.1.
- Real transition clips form the "not a sign" class.
- Daily words oversampled.
- EMA 0.998.

Selection is by mean new-signer MRR on the val signers (TTRS, TSL-ONE-S, th_sl, AGENT).

3.2 One-pass spotting [v7]

v6 encoded every candidate window separately (hundreds of encoder passes per utterance). v7 encodes the utterance **once**:

1. Encoder → frame states H ($T \times 768$).
2. **Span head** (modules/spanhead.py): for each window [a, b), the mean, start, end and attention-pooled states (from float64 cumulative sums) feed a small MLP that gives the window embedding. It is distilled from the isolated-window encoder (cosine + 0.2 · InfoNCE) on synthetic and real sentences.

3. Bank score per window = mean of the top-2 cosine similarities per concept (allowed concepts only in conversation mode).
4. A semi-Markov decoder combines window scores with the tagger's $P(\text{begin}) / P(\text{inside})$, a length cost λ and a minimum length. Tempo normalisation applies to fast signing.
5. Post-processing: drop segments that are too short or have low $P(\text{sign})$ (hand transitions); penalise immediate repeats.
6. Calibration: logistic $P(\text{correct})$ from score, z-score, top-1 margin, gap to the null prototype, $P(\text{sign})$, window agreement, support ratio and length $\rightarrow$ accepted / uncertain / unknown.

3.3 Frame tagger

A BiGRU (2×192 , bidirectional) over $[H \parallel 13 \text{ kinematic features}]$ gives non-sign / begin / inside (1.26 M parameters). Training uses synthetic sentences (clips joined with real transitions, at speeds 0.7–2.0) plus aligned TSL51 tune templates.

- **[v7]** The final tagger uses all tune templates; its epoch is chosen on a synthetic held-out set.
- **[v7]** Two fold taggers (each blind to half the templates) give out-of-fold probabilities, so the decoder is tuned on templates the tagger never saw.

3.4 Face channel [v7]

52 MediaPipe blendshapes plus head pose (`modules/face.py`):

- **Negation:** head-shake cycles.
- **Emotion:** smile, frown, brow and mouth blendshapes against per-emotion thresholds calibrated on 580 neutral dictionary faces. It sets the voice style.
- **Question cues** (outer-brow raise, furrow) are computed but **not used** to change the sentence, because they did not separate questions on real phrases (`result_v7.md §7`).

3.5 Language layer

Accepted words plus face cues go to gpt-4.1-mini with an evidence guard: it may only use the accepted words, it may reorder them from TSL order to Thai, and it must leave out what is uncertain. A rules composer (time words first, question words last) is the offline fallback. Two adjacent uncertain words withhold the sentence. Only a confirmed sentence is spoken. A withheld reading is shown as "(ไม่แน่ใจ) ..." with each word as `word?` or `[?]  $\approx$  nearest`.

3.6 Vocabularies

- **conversation** (default, 322 concepts): `vocab/daily_conversation.json` $\cup$ TSL51 words that are in the bank. 18 daily words have no Thai data, ไหม included; they are covered by `vocab/missing_word_strategy.json` (no foreign signs).
- **full:** all 9,922 concepts.

New words or signers are added without retraining by appending embeddings to `models/bank.npz` (`models/README.md`).

4. Deployment

piece	technology
model files	<code>models/</code> (<code>.pt</code> for PyTorch, <code>onnx/*.onnx</code> for production, <code>span_head.npz</code> , bank, settings)
runtime	<code>modules/runtime.py</code> : one NumPy interface over ONNX Runtime (default) or PyTorch
service	<code>docker-compose.yml</code> : gateway <code>nginx</code> → <code>web</code> (Next.js 16 standalone) · <code>sign-api</code> (FastAPI, ONNX Runtime, MediaPipe for uploads) · <code>tts</code> (MMS-TTS Thai VITS, voice baked into the image); models mounted read-only from <code>\${THAISLM_MODELS_DIR:-./models}</code>
scaling	stateless API → <code>docker compose up --scale sign-api=N</code> ; <code>nginx</code> resolves the Docker DNS name per request
browser	<code>webapp/web/lib/holistic.ts</code> runs MediaPipe on the GPU/CPU of the device; live mode starts on hands-up (250 ms), ends after 900 ms hands-down, prerolls 500 ms, caps at 20 s

Latency (laptop CPU, after landmarks): 0.28 s for 2.4 s of signing, 0.77 s for 10 s. Server-side MediaPipe for an uploaded video adds 4–9 s, which is why the browser does it.

5. GPU jobs and cost control

- Only the encoder needs a data-center GPU. Everything else (MediaPipe extraction, harvest, span head, taggers, tuning, export) ran on the laptop.
- `GCP/script/vertex.py launch` submits the same job to `asia-southeast1`, `us-central1` and `eu-west4` (Spot A100). The first to start wins and the others are cancelled while pending (not billed).
- Inside the job, data and models are copied from GCS to local disk with the GCS client (no bucket-mount reads during training).
- A persistent loader and a 1.2 h timeout bound the cost.
- `python main.py cloud cleanup` empties the bucket, deletes every job, and checks that no VM, disk, IP, snapshot, image, endpoint or model is left.